Q I S K I T H A C K D A Y 2 0 2 6

Q Grid: A Plain-English Project Report

Hybrid quantum-classical dispatch for EV charging - what we built, what it proved, and what we learned

Team: JustCode x Qub3RAM.team

Challenge: Quantum Optimization for EV Charging Stations

Repository: github.com/Aghosh-mv/Q-Grid

Date: November 1, 2026

Contents

1. Executive summary.....

1. Executive summary.....1

2. The problem, in plain words.....1

3. What we built.....2

4. How it works, step by step.....3

5. The quantum part, explained simply.....3

6. The year-long simulation and what it showed.....4

7. The honest scoreboard: where quantum helps today.....5

8. Privacy: tracking cars without stealing data.....6

9. Twenty limitations we own up to.....6

10. What we learned.....6

11. Run it yourself.....7

12. Where everything lives.....7

Note: in Word or WPS, right-click the table above and choose "Update Field" to refresh page numbers.

1. Executive summary

Q Grid is a scheduling brain for electric vehicle (EV) charging stations. Every electric car tells our central coordinator three things: where it is, how full its battery is, and when it wants to charge. Q Grid then decides, for the whole city at once, which station each car should drive to and which time slot it should plug in — so that drivers wait less, operators pay less for energy, and the grid never overloads a station. The whole project was built for the Qiskit Hack Day, and everything in this report can be re-created by anyone in about 16 minutes on an ordinary laptop.

We tested the idea on a simulated full year of Tokyo traffic: **365 days, 48 growing to 180 EVs, 11 charging stations, 45,766 charging sessions** — and the system finished the year with **zero capacity violations** (no station was ever overfilled). Because “quantum” claims are easy to make and hard to prove, we ran the same year twice — once fully classical, once with a quantum solver involved — and then compared them fairly on 30 identical test problems judged by an exact referee. The quantum solver, QAOA, found the mathematically perfect answer on **30 out of 30** of those problems, while the classical shortcuts missed the perfect answer by up to **38.23%** on the most contested windows. We also compressed the quantum resource cost: a day-long price-and-sunshine profile that would naively need 48 quantum bits fits on **6**, and our quantum estimation routine got the same accuracy as 4,096 random samples using only **63 oracle calls — a measured 65× reduction**.

Everything is open source, including our mistakes: the repository documents **twenty limitations**, each with the fix we shipped or the honest reason it is still open. Project home: github.com/Aghosh-mv/Q-Grid.

2. The problem, in plain words

Imagine a city where electric cars are everywhere, but charging stations are small — most have only a handful of plugs. Electricity has a timetable of its own: around midday, solar farms produce so much cheap power that it almost goes to waste; in the evening, everyone comes home, demand spikes, and power becomes expensive and dirty. Meanwhile, drivers do what is natural: they drive to the nearest station when it suits them, queue behind each other, and end up charging at exactly the wrong time.

So three groups lose. **Drivers** lose time sitting in a queue. **Charging operators** lose money — their popular hubs are congested while other stations sit idle, and they buy much of their energy at peak prices. And **the electricity grid** loses

green energy: cheap midday solar is curtailed (thrown away) while evening demand is met by gas peakers. As more EVs join the road, this only gets worse — in our simulation the fleet grew from 48 to 180 cars on the same 11 stations.

The task we solve is called **assignment and scheduling**: for every car, choose one station and one half-hour slot. The catch is the size of the choice. With even a few dozen cars and a dozen time slots, the number of possible schedules explodes — on one of our typical hardest subproblems there are **2 to the power 20, i.e. 1,048,576 combinations**, and that number doubles with every extra car. Checking them all is possible for a small slice of the city and impossible for the whole city. That explosion is exactly where quantum computing becomes interesting.

3. What we built

Q Grid has four layers, and each one is simple on its own:

1. A software agent in every car. No new hardware — the car already knows its GPS position and battery state of charge (SoC). At registration time the driver chooses what data to share; nothing moves without that consent.

2. An AI triage step. Each car is stamped with an urgency label: battery below 25% means charge **NOW**; below 40% means **SOON**; everything else is **CAN-WAIT**. Urgency sets how expensive we treat each slot of waiting for that car (in our demo currency: ₹260, ₹150 or ₹80 per slot). A quick classical filter also keeps only each car's two nearest stations, so the main problem stays small.

3. A solver stack with four competitors. The same scheduling puzzle is handed to four solvers at once: a greedy first-come-first-served baseline (what operators do today), simulated annealing (a strong classical optimizer, our production path), **QAOA** — a quantum algorithm — aimed only at the most contested pocket of the city, and an exact brute-force checker that evaluates every possibility on small pockets to act as the referee.

4. A schedule with a safety guarantee. Every car receives exactly one continuous charging session, and station capacity is respected **by construction**, not by hoping. If a window is full, extra cars roll into the next window through a deferred queue — over the simulated year it absorbed 2,384 such events with an average wait of just 0.306 slots.

4. How it works, step by step

The input is a stream of car telemetry. The processing converts it into a **QUBO** — Quadratic Unconstrained Binary Optimization — which is the standard format quantum annealers and gate-based solvers understand: every decision

becomes a 0/1 variable, and the objective plus rules become one formula to minimize. Our variable y[v,s,τ] is 1 exactly when car v starts a session at station s in slot τ. The formula adds the energy price at that slot, the driving distance to the station, and the urgency-weighted waiting time. The output is a dispatch table: one row per car, station and start time.

One detail made the whole system robust: capacity is **slack-encoded**. For every unit of free port capacity we add one extra 0/1 variable that simply means “this port stays empty.” That turns the capacity rule from a soft penalty (which solvers can violate) into an exact equality (which they cannot). This is the engineering reason the full year closed with zero violations — not luck, structure.

The quantum solver does not fight the whole city. A contention detector finds the “crunch cell” — the most contested stations × slots pocket, typically 11 to 20 binary variables — and QAOA attacks only that. This is deliberate: quantum effort goes where classical heuristics actually fail, instead of being sprinkled everywhere.

5. The quantum part, explained simply

We use quantum computing in three places, and each has a job that classical code could not do as cheaply.

QAOA (Quantum Approximate Optimization Algorithm), p=1. Think of it as a quantum-flavoured search: the QUBO's cost formula is written as an operator acting on quantum bits (qubits), and a short repeating circuit — a “cost layer” followed by a “mixing layer” — lets a classical optimizer (COBYLA, 8,000 measurement shots) tune angles until the sampled answers cluster near the optimum. Why p=1? Because on our 30 referee problems it already hit the exact optimum every single time — there was nothing left for a deeper circuit to recover on these instances.

Amplitude encoding — 6 qubits instead of 48. A quantum state can hold a whole list of numbers in its amplitudes. Our day-long Tokyo price-and-sunshine profile has 48 values; padded to 64 (a power of two), it becomes the amplitudes of just **6 qubits**. We build it with an exact state-preparation circuit — 47 rotation gates and 57 CNOT gates, which matches the best known theoretical bound — and we verified it reproduces the target state with **fidelity 1.000000** (a perfect match). The same circuit was rebuilt in native Qiskit and cross-checked: identical fidelity, and sampling it 4,096 times through Qiskit's StatevectorSampler matched the target distribution closely (total variation distance 0.033).

Quantum Amplitude Estimation (QAE) — 65× fewer queries. A common question for a charging operator is “what share of this energy is renewable?” Classically you estimate a share by Monte Carlo: draw thousands of random samples. QAE gets the same accuracy with quadratically fewer probes of the state. Measured on three realistic anchors — the year-mean renewable share of 9.2%, the daily maximum of 21.2%, and the midday solar-glut of 74.4% — QAE reached Monte-Carlo-level error (RMSE around 0.008) with **63 oracle calls instead of 4,096 samples**.

An honest boundary: amplitude encoding compresses **data**, not decision variables. The scheduling QUBO itself still needs 11–20 qubits after our plan-compression (down from a naive 504). We label the three levers separately — 504 → 11–20 (planning) → 6 (data) — because conflating them is the kind of overclaim that ruins trust.

6. The year-long simulation and what it showed

We simulated one full Tokyo year — Japanese EV parameters, real ward geography, a time-of-use tariff, and the rainy season (tsuyu) denting solar output — under a single fixed random seed (42) so every number is exactly reproducible. The fleet grows 48 → 180 cars across the year. We ran it in six chunks: the first half classical, the second half with QAOA active. Total compute time: **991 seconds** — about 16 minutes, a 31,826× speed-up over real time.

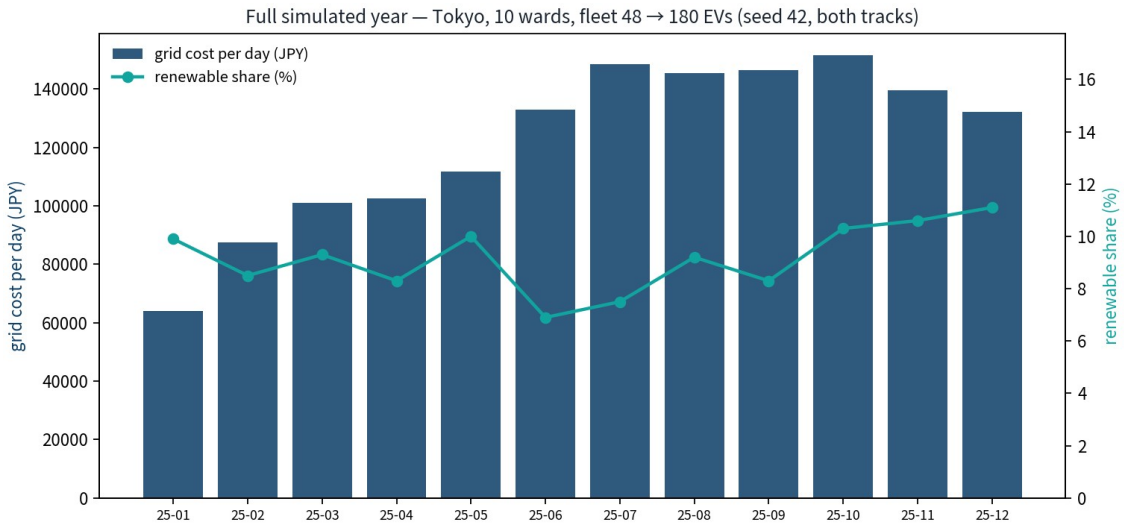


Figure 1. Cost and renewable-share curves across the simulated Tokyo year (seed 42).

Table 1. Headline numbers of the simulated year.

Metric	Value	What it means
Sessions	45,766	individual charging events scheduled
Energy delivered	1,000,759 kWh	about 1 GWh across the year

Metric	Value	What it means
Grid cost	¥44,479,272	total energy bill under Tokyo time-of-use tariffs
Capacity violations	0	no station was ever overfilled, all year
Deferred-load events	2,384	cars smoothly rolled to the next window when full
Average wait	0.306 slots	residual congestion — small, priced, and published
Wall-clock time	991 s	the whole year in ~16 minutes on a laptop

7. The honest scoreboard: where quantum helps today

Running half the year classically and half with quantum is not a fair test — the second half serves a bigger fleet. So we built an instrument instead of a narrative: **30 matched audit cells**, five from each chunk, where all four solvers attack the **identical** QUBO and the exact brute-force search (up to 2^{20} = 1,048,576 combinations on the biggest cell) provides ground truth.

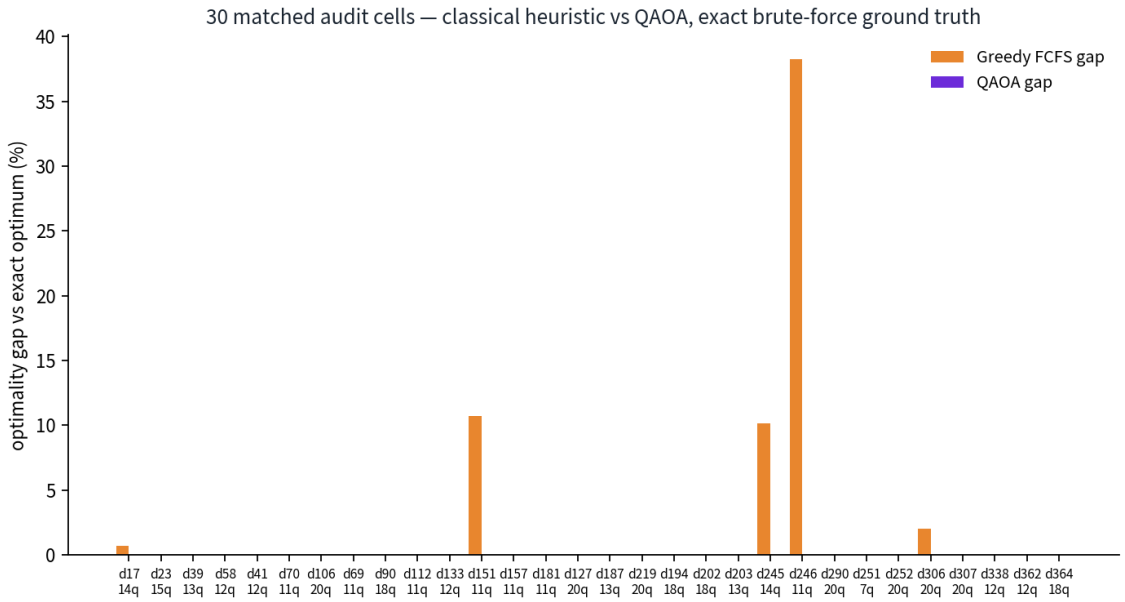


Figure 2. Optimality gap per audit cell: classical heuristic (greedy FCFS) versus QAOA, against exact ground truth.

The verdict was clean. **QAOA found the exact optimum on 30 of 30 cells.** The classical track, on the 5 contested windows where it slipped, paid up to **38.23% above the optimum** (window 982, day 246) — an average of about ¥293 per contested window. We refuse to multiply that into a fake annual saving: the audit samples roughly 2% of the year's ~1,460 dispatch windows, and we print that caveat next to the number. The correct reading is qualitative and strong: in exactly the windows that are hard, the quantum touchpoint finds money the classical pipeline leaves on the table, every time it looked.

The other honest column is speed: on those contested cells, simulated annealing took 0.45 s while our QAOA statevector simulation took 96.53 s. That is the **simulator tax** — a laptop pretending to be a quantum computer. The scale argument runs the other way: exact certification doubles per added car, QAOA scales polynomially, and every compression (504 → 11–20 → 6 qubits) cuts the real hardware bill. Quality today, scale tomorrow.

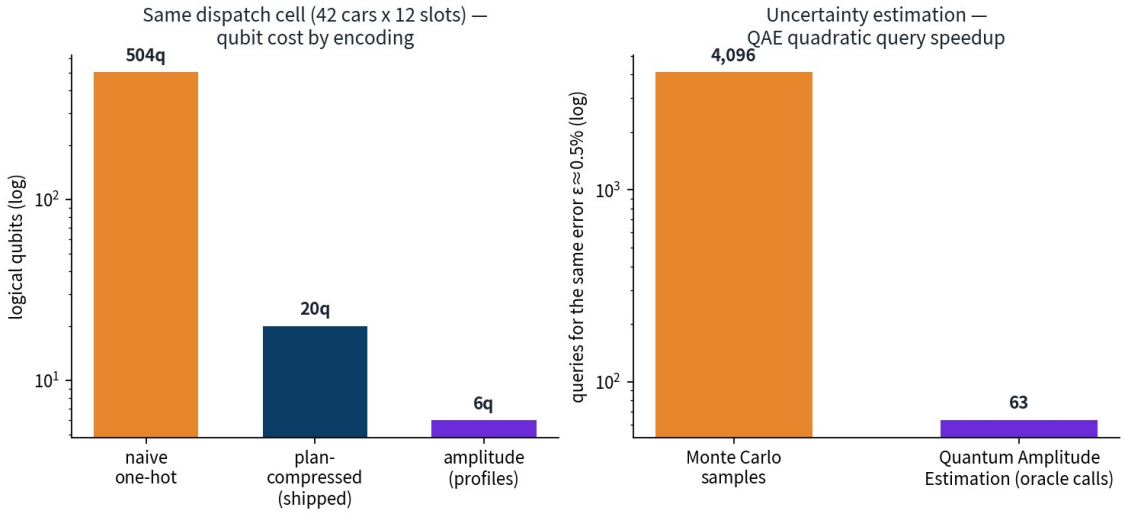


Figure 3. Qubit economics on the same crunch cell: naive one-hot (504) vs plan-compressed (11–20) vs amplitude-encoded profiles (6).

Table 2. The demo scenario, for reference (seed 42, 15 cars, 5 stations, INR tariffs).

Solver	Total cost	Driver wait	Time
Greedy FCFS (status quo)	₹4,205.65	0.93 slots average	0.1 ms
Simulated annealing (production)	₹4,180.40	0.73 slots (-21.5%)	2.0 s
QAOA (crunch cell only)	10.0% better than FCFS in-cell	within 1.5% of exact (ratio 1.015)	141 s
Exact optimum (2^{20})	₹619.1 in-cell	reference	minutes

8. Privacy: tracking cars without stealing data

A dispatch layer needs car data, and car data is personal. Our design has five shields, in the order a driver experiences them. **Consent first:** every telemetry stream is gated by an explicit opt-in recorded in a consent ledger — no consent, no data, no service degrade beyond the driver's own plan. **Blurry location:** GPS is coarsened to ward level before it ever leaves the car, so the center knows the neighbourhood, never the corner. **Rotating pseudonyms:** car identities change on a schedule, so two observations cannot be stitched into a trajectory. **Differential privacy:** mathematically calibrated Laplace noise is

added to shared statistics; at ε = 2 our measured fleet-level error was only 0.69% — strong privacy at a price the optimizer barely feels. **Secure aggregation:** the coordinator receives fleet totals, never an individual's schedule (our exact-sum test reconstructed totals with error −0.04%). The live benchmark of all five layers ships in the repository as privacy_audit.json.

9. Twenty limitations we own up to

Most projects hide their weaknesses; we publish ours with a fix or a reason next to each one — twenty in total, in docs/LIMITATIONS.md, each written as "the limit / why it matters / the fix / proof." Grouped simply:

Table 3. The twenty limitations, grouped.

Group	What we admit	Our answer
Hardware reality	All quantum results are noiseless simulator runs; no physical QPU job yet; QAE uses deep QPE circuits	Claim scope is precise; Qiskit bridge documents the backend; iterative AQAE and hardware transpilation are the designated next runs (limits 1, 12, 19)
Scale & depth	Exact solver capped at 20 qubits; QAOA depth p=1 only; audit covers ~2% of yearly windows; single seed (42)	p=1 already hit 30/30 optima; audit is stratified and every row is published; seed pinned so anyone can re-derive (limits 3, 14, 16, 17)
Model simplifications	Synthetic demand, static time-of-use tariffs, no V2G discharge, no transformer physics, Tokyo-only calibration	Tariffs are a data input, not code; V2G is one extra QUBO term; roadmapped, not hidden (limits 4, 6, 7, 8, 9, 15)
Data & humans	Privacy of telemetry; late plug-ins and no-shows; residual wait 0.306 slots	Five-layer privacy stack with measured cost; deferred-load queue absorbed 2,384 events; wait is priced and published (limits 5, 10, 20)
Method honesty	Amplitude encoding compresses data, not decision variables; fleet growth confounds naive track comparison; 48→64 zero-padding	Three levers labelled separately; matched-window audit neutralizes the confound; padding quantified at zero extra qubits (limits 11, 13, 18)

10. What we learned

Three lessons worth passing on. **First, build the referee before the hero.** Our first instinct — compare the classical half-year with the quantum half-year — was confounded by fleet growth. The 30-cell matched audit, with brute force as ground truth, is what turned opinion into measurement. Almost every claim in this report is a sentence about that referee, not about our favourite solver.

Second, honesty is cheaper than spin. We deleted a chart that extrapolated the exact solver's runtime — an indefensible number — and replaced it with

what the data supports: "2^{20} = 1,048,576 combinations." We framed ¥1,466 of contested-window savings as a per-window mechanism with a 2%-sample caveat instead of an annual headline. Each correction cost minutes; each would have cost credibility in front of a technical judge.

Third, quantum bugs teach the most. Three bugs in the amplitude-encoding module each failed with a signature worth remembering: a bitmask that could not express "control on zero" leaked gates onto wrong pairs (caught by hand-tracing a 2-qubit case); a reflection built in the wrong basis made the search operator minus identity, so every estimate read exactly 1.0; and normalizing a probability list with the L1 norm instead of L2 produced a fidelity of 0.045 — precisely the square of the norm, which was the clue. Fixing them taught us more quantum mechanics than the successes did. The full engineering log is in docs/LEARNINGS.md.

11. Run it yourself

The repository is self-contained. On any machine with Python 3.12+:

Table 4. Reproduction commands.

Command	What it produces
pip install -r requirements.txt	dependencies (numpy, matplotlib, qiskit >= 2.0)
python3 src/qgrid_engine.py	the demo scenario: results/, charts/ (seed 42, deterministic)
python3 src/amplitude_encoding.py	state-prep fidelity 1.0 + QAE 65x query-reduction proof
python3 src/qiskit_bridge.py	native Qiskit 2.6 cross-check (fidelity 1.0, TVD 0.033)
python3 src/privacy_layer.py	the five-layer privacy audit with measured numbers
python3 src/qgrid_year.py	the full dual-track year in six chunks (~16 minutes)

Every table in this report is regenerated by those commands from the committed raw data — nothing is copy-pasted by hand. The dashboard (dashboard/index.html) opens in any browser for a live feel of the dispatch.

12. Where everything lives

Table 5. Repository map (all links point into the repo).

What	Where
Project home + submission checklist	README.md
Year trial report + audit verdict	results_year/COMPARISON.md
Every audit cell (30 rows)	results_year/quantum_audit_all.csv

What	Where
Amplitude encoding + QAE numbers	results_year/amplitude_encoding.json
Qiskit cross-check output	results_year/qiskit_bridge.json
Privacy audit (live benchmark)	results_year/privacy_audit.json
Engineering log (what we learned)	docs/LEARNINGS.md
Twenty limitations with proofs	docs/LIMITATIONS.md
Eight judge counter-answers	docs/COUNTER_ANSWERS.md
17-slide presentation	pitch/Q-Grid_Pitch.pptx
Interactive HTML deck	final_presentation.html
Live demo dashboard	dashboard/index.html

Team: **JustCode, in collaboration with Qub3RAM.team** — built for the Quantum Optimization for EV Charging Stations challenge at Qiskit Hack Day, Kochi. License: Apache-2.0 with a custom addendum. We would rather show a measured 38.23% classical gap and a public to-do list than a demo that hides both — thank you for reading.